Assistant Chat Screen and RAG Technical Overview

Purpose of This Report

This report explains the end-to-end technical process of the ElderMeds assistant chat screen, why an LLM is used, how Retrieval-Augmented Generation (RAG) works in this project, and the main technical terms used in the implementation.

The assistant is not just a visual chat box. It is a full workflow that connects the React Native mobile interface, authenticated Node/Express backend, PostgreSQL health records, Python FastAPI ML service, and a local LLM through Ollama/Llama 3 style endpoints.

Why an LLM Is Used

The assistant uses a Large Language Model (LLM) because users can ask questions in natural language instead of using fixed forms or buttons.

Example user messages:

```
Did I miss any medicine this week?
Which medicines are running low?
I feel lonely today.
How can I reduce this risk?
Add 20 tablets to Metformin stock.
```

A traditional rule-based chatbot would require many manually written patterns for every possible sentence. An LLM can understand flexible wording, identify the user's intent, generate database queries for record-based questions, and produce readable answers.

This is especially useful for elderly users and caregivers because they do not need to know exact app terminology. They can ask questions in a natural way, and the system can still route the request to the correct workflow.

Technical Justification for the LLM

The LLM is justified for these technical reasons:

1. Natural language understanding: It converts human language into structured intent.
2. NL2SQL generation: It can translate healthcare-record questions into SQL queries.
3. Personalized response generation: It explains retrieved user-specific records in simple language.
4. Follow-up generation: It suggests useful next questions based on the previous answer.
5. Caregiver-aware wording: It can adapt answers for elder or caregiver role.
6. Reduced UI complexity: Users do not need multiple filters and forms for every possible query.
7. Better accessibility: Voice and text questions are easier than navigating many screens.

The important design decision is that the LLM is not trusted directly. It does not access

the database by itself and it cannot execute SQL. The backend validates all generated SQL before execution.

What RAG Means

RAG means Retrieval-Augmented Generation.

In simple words:

```
Retrieve trusted information first.
Then generate an answer using only that retrieved information.
```

Without RAG, an LLM may answer from general training knowledge and may hallucinate. Hallucination means the model invents information that is not supported by real data.

With RAG, the system first retrieves relevant records from PostgreSQL, then asks the LLM to explain those records. Therefore, the final answer is grounded in the user's own health data.

In this project, the RAG flow is:

```
User question
-> ML-service converts question to SQL
-> Backend validates SQL
-> Backend retrieves user-specific records from PostgreSQL
-> Sensitive values are redacted
-> ML-service generates a final answer from retrieved rows
-> Backend saves the conversation
-> Frontend displays the assistant response
```

Main Technical Terms

LLM: Large Language Model. A model that understands and generates human language. In this system it helps understand questions, generate SQL, and write friendly answers.

RAG: Retrieval-Augmented Generation. A pattern where the system retrieves trusted data first and then uses an LLM to generate an answer from that data.

NL2SQL: Natural Language to SQL. The process of converting a user question into a SQL query.

SQL: Structured Query Language. The language used to read records from PostgreSQL.

PostgreSQL: The relational database that stores user records such as medicines, routines, medication status events, allergy cards, mood logs, caregiver alerts, and assistant messages.

Schema digest: A controlled description of the database tables and columns that the LLM is allowed to use. It prevents the LLM from guessing unknown or unsafe tables.

Intent: The detected purpose of the message. Example intents include missed medications, low stock, diabetes prediction, health advice, emotional support, or agentic action.

User scoping: A safety rule that every user-specific database query must include the current authenticated user id. In this codebase, the backend binds the logged-in user's id as $1.

Fallback: A safe backup response returned when the assistant cannot answer safely or when an external dependency fails.

Hallucination: An incorrect or invented answer from an LLM. RAG reduces this by forcing answers to be based on retrieved records.

Redaction: Removing or replacing sensitive values before sending data to the LLM. Examples include passwords, tokens, emails, and phone numbers.

Agentic action: A chat request that changes something in the system, such as adding medicine stock or updating routine time. These actions require confirmation before execution.

JWT: JSON Web Token. The authentication token used by the frontend when calling protected backend APIs.

API: Application Programming Interface. The HTTP endpoints used by frontend, backend, and ML-service to communicate.

Main Files in the Implementation

Frontend:

```
frontend/App.js
frontend/src/screens/AssistantChatScreen.js
frontend/src/components/ChatBubble.js
frontend/src/services/assistantService.js
frontend/src/services/apiConfig.js
frontend/src/services/authService.js
```

Backend:

```
backend/src/index.js
backend/src/routes/assistant.js
backend/src/controllers/assistantController.js
backend/src/services/assistantOrchestrator.js
backend/src/services/sqlValidator.js
backend/src/services/assistantAgenticActionService.js
backend/src/data/schemaDescription.js
backend/migrations/1746590400000_assistant_tables.js
```

ML-service:

```
ml-service/app/routers/assistant.py
ml-service/app/services/nl2sql.py
ml-service/app/services/rag_answer.py
ml-service/app/services/health_advice_answer.py
ml-service/app/services/ollama_client.py
ml-service/app/prompts/nl2sql_system.txt
ml-service/app/prompts/answer_system.txt
```

End-to-End Flow Summary

The complete assistant runtime path is:

```
React Native Chat Screen
-> Frontend assistant service
```

```
-> Node/Express assistant route
-> Auth and rate limit middleware
-> Assistant controller
-> Assistant orchestrator
-> Optional deterministic workflow
-> Optional ML-service NL2SQL
-> SQL safety validator
-> PostgreSQL query
-> Optional ML-service answer generation
-> Conversation persistence
-> Response returned to frontend
-> Chat bubble rendered to user
```

Step 1: Opening the Chat Screen

The assistant opens from frontend/App.js. The openAssistant function stores the screen the user came from, optionally stores an initial prompt, and changes activeScreen to assistant-chat.

When activeScreen is assistant-chat, App.js renders AssistantChatScreen and passes:

```
initialPrompt
onBack
onAgentNavigate
user
```

The chat screen uses the user role to decide whether to show elder wording or caregiver wording.

Step 2: Chat Screen State

AssistantChatScreen.js keeps local state for:

```
messages: all visible user and assistant bubbles
input: current text box value
conversationId: current saved conversation id
sending: whether a request is in progress
error: visible error message
isListening: whether speech recognition is active
speakingId: which assistant message is being read aloud
showHistory: whether the past-chat modal is visible
conversations: loaded chat history list
followUpsCollapsed: whether suggested follow-up questions are hidden
assistantProgressIndex: progress stage while waiting for answer
```

If an initial prompt is provided, the screen automatically sends it once. This is used when the floating assistant button starts the chat with a summary question.

Step 3: User Sends a Message

The user can send a message in four ways:

1. Type in the text box and press Send.
2. Tap an initial suggested prompt.
3. Tap a follow-up question after an assistant reply.

4. Speak through the microphone button.

All of these paths call the same sendMessage function.

sendMessage does the following:

```
Trim the message
Ignore empty messages
Stop current speech playback
Clear previous errors
Set sending = true
Add the user bubble immediately
Clear the input box
Scroll to bottom
Call assistantService.chat
```

Step 4: Frontend API Request

frontend/src/services/assistantService.js sends:

```
POST /assistant/chat
```

The base URL comes from apiConfig.js. In development it can use an explicit EXPO_PUBLIC_API_URL, the Metro host, or a default localhost/LAN address.

The request body is:

```
{
  "message": "Did I miss any medicine this week?",
  "conversationId": 12
}
```

The authService adds:

```
Authorization: Bearer <jwt>
```

Step 5: Backend Route and Middleware

backend/src/index.js mounts the assistant route at:

```
/api/assistant
```

backend/src/routes/assistant.js applies:

```
requireDatabase
requireAuth
rateLimitChat
```

requireDatabase ensures the database connection is available.

requireAuth verifies the JWT and attaches the authenticated user to req.user.

rateLimitChat limits the number of chat requests per user per minute. The default is 30 requests per minute.

Step 6: Assistant Controller

assistantController.js receives the request and calls:

```
handleChat({
  userId: req.user.id,
  role: req.user.role || 'user',
  message: req.body?.message,
  conversationId: req.body?.conversationId ? Number(req.body.conversationId) : nul
l,
});
```

This means the backend, not the frontend, decides the real user id. That is important for security.

Step 7: Assistant Orchestrator Classification

assistantOrchestrator.js is the main brain of the chat feature. It first classifies the message before deciding whether to use the general RAG pipeline.

The main branches are:

```
Pending confirmation
Agentic action
Emotional support check-in
Health risk advice
Stroke prediction
Hypertension prediction
Diabetes prediction
General healthcare-record RAG question
```

This hybrid approach is important. Not every question should go through generic NL2SQL. Some workflows are safer and more accurate as deterministic service calls.

Step 8: Agentic Action Flow

Agentic actions are messages that change saved data.

Examples:

```
Add 20 tablets to Metformin stock.
Set breakfast routine to 8 AM.
Delete Metformin.
Send a refill alert for Paracetamol.
```

The backend parses the action and asks for confirmation first:

```
Add 20 tablets to Metformin stock. Please confirm before I make this change.
```

Only if the user replies yes, confirm, or similar wording does the backend execute the change.

Supported action domains include:

```
medication create/update/delete
```

```
routine update
stock refill/set_stock
caregiver alert create/mark_read/delete
```

After a successful action, the backend can return navigation data so the frontend moves the user to the affected screen.

Step 9: Emotional Support Flow

If the message expresses emotional need, such as "I feel lonely today", the backend runs an emotional support check-in.

It uses:

```
emotionAnalysisService
interventionService
alertService
sessionRepository
activityRepository
alertRepository
```

The result can include:

```
detected emotion
confidence score
stress score
loneliness score
risk level
intervention message
cognitive activity recommendation
caregiver alerts created
```

This branch saves the interaction as a health-related check-in, not just a chat answer.

Step 10: Health Risk Prediction Flow

Messages about checking risk are routed to specific prediction services:

```
diabetesPredictionService
strokePredictionService
hypertensionPredictionService
```

The backend extracts values from text where possible.

Examples:

```
My sugar is 150 today.
My BP is 145 over 90.
Now my weight is 60kg.
```

The prediction service prepares model input, calls the ML prediction endpoint, creates a summary, saves prediction history, and saves the chat messages into assistant_conversations and assistant_messages.

Step 11: Health Advice Flow

Questions like "How can I reduce this risk?" are handled by healthAdviceService.

This service retrieves trusted health advice chunks, uses the latest saved prediction context, and may call the ML-service health-advice endpoint.

The answer includes a safety note:

```
This is general health guidance, not a medical diagnosis.
```

This is separate from general NL2SQL because advice should come from a controlled health advice knowledge base, not only raw user records.

Step 12: General RAG Question Flow

If the message is a general healthcare-record question, the system uses the two-stage RAG pipeline.

Example questions:

```
Did I miss any medicine this week?
Which medicines are running low?
Show my recent caregiver alerts.
Are any saved medicines dangerous for me?
How has my mood been recently?
```

The backend first ensures there is a conversation. If no conversation id exists, it creates one in assistant_conversations. The conversation title is based on the first message.

Then it loads recent messages from assistant_messages so the LLM has short conversation context.

Step 13: NL2SQL Stage

The backend calls the ML-service:

```
POST /assistant/nl2sql
```

The payload includes:

```
question
user_role
recent_messages
schema_digest
```

The schema digest comes from backend/src/data/schemaDescription.js. It lists the only database tables and columns the LLM is allowed to use.

The ML-service uses nl2sql.py. It may first try rule-based SQL for common questions. If no rule-based match exists, it calls the LLM with:

```
NL2SQL system prompt
user question
recent messages
schema digest
spaCy preprocessing hints
```

The expected output is JSON:

```
{
  "sql": "SELECT ... WHERE user_id = $1 LIMIT 100",
  "params": ["$1"],
  "intent": "missed_medications",
  "fallback": false,
  "fallback_message": ""
}
```

Step 14: SQL Safety Validation

The backend never executes LLM SQL directly.

sqlValidator.js enforces these rules:

```
Exactly one statement
Only SELECT queries
Only allowed tables from schemaDescription.js
No INSERT, UPDATE, DELETE, DROP, ALTER, CREATE, or other write commands
No SQL comments
No pg_* catalog access
No information_schema access
Every user-scoped table must include user id = $1
Only $1 parameter is allowed
LIMIT 100 is added if missing
```

If validation fails, the user receives a safe fallback answer:

```
I cannot answer that safely.
```

The failure reason is stored for debugging and evaluation.

Step 15: Database Retrieval

After validation, the backend executes the SQL using PostgreSQL.

The query is always bound with:

```
$1 = authenticated user id
```

The backend also sets a statement timeout, defaulting to 5 seconds, and limits returned rows to 100.

This protects performance and prevents long-running generated queries.

Step 16: Redaction Before Answer Generation

Before retrieved rows are sent back to the LLM for explanation, sensitive values are redacted.

Sensitive keys include:

```
password
password_hash
token
```

```
authorization
caregiver_phone
phone
email
caregiver_email
```

This reduces privacy risk because the LLM only receives data needed to answer the question.

Step 17: Answer Generation Stage

The backend calls:

```
POST /assistant/answer
```

The payload includes:

```
question
validated SQL
retrieved rows
intent
user_role
```

The ML-service uses rag_answer.py and answer_system.txt to generate a friendly response. The answer generator is instructed to use only the retrieved rows and avoid inventing facts.

The expected response is:

```
{
  "answer": "You missed two medicine doses this week...",
  "follow_ups": [
    "Which medicine did I miss most?",
    "What should my caregiver monitor?"
  ]
}
```

Step 18: Conversation Persistence

The system stores chat history in two tables:

```
assistant_conversations
assistant_messages
```

assistant_conversations stores:

```
id
user_id
title
created_at
updated_at
```

assistant_messages stores:

```
id
conversation_id
user_id
role
content
```

```
sql_used
rows_returned
intent
fallback_reason
latency_ms
created_at
```

This means the app can show previous chats, reload messages, inspect what SQL was used, and track fallback reasons.

Step 19: Frontend Response Handling

The backend returns data like:

```
{
  conversationId,
  answer,
  sql,
  rows,
  sources,
  safetyNote,
  fallback,
  followUps,
  actionResult,
  pendingAction,
  intent
}
```

AssistantChatScreen.js then:

```
Updates conversationId if needed
Creates an assistant bubble
Stores SQL and rows on the message object
Shows trusted sources or safety note if available
Displays follow-up suggestions
Stops loading state
Scrolls to the bottom
Optionally navigates after action completion
```

Step 20: Chat Bubble Rendering

ChatBubble.js displays:

```
User messages on the right
Assistant messages on the left
Assistant avatar
Fallback styling for safe-error replies
Trusted sources if available
Safety note if available
Listen button for text-to-speech
```

The SQL and rows are kept in the message object for traceability, even if the current bubble UI mainly displays the readable answer and source labels.

Step 21: Voice Input and Output

Voice input uses expo-speech-recognition.

Flow:

```
User taps Talk
App requests microphone permission
Speech recognition starts
Interim transcript appears in input box
Final transcript is submitted to sendMessage
```

Voice output uses expo-speech.

Each assistant message can be read aloud using the Listen button. If the same message is already speaking, tapping again stops it.

This improves accessibility for elderly users.

Step 22: Chat History Management

The chat screen supports persistent history.

Endpoints:

```
GET /api/assistant/conversations
GET /api/assistant/conversations/:id/messages
PATCH /api/assistant/conversations/:id
DELETE /api/assistant/conversations/:id
```

The user can:

```
Open a previous chat
Start a new chat
Rename a chat
Delete a chat
```

When an old chat is selected, messages are mapped from database rows into frontend bubble objects.

Security and Safety Design

The system has several safety layers:

1. JWT authentication ensures only logged-in users can call assistant APIs.
2. The backend uses req.user.id, not a user id from the frontend body.
3. Chat requests are rate limited.
4. LLM-generated SQL is validated before execution.
5. Only SELECT queries are allowed in the RAG SQL path.
6. Every user-specific query must include the authenticated user id.
7. Sensitive fields are redacted before LLM answer generation.
8. Database statement timeout prevents long-running queries.
9. Fallback responses are returned when safety checks fail.
10. Medical advice includes a non-diagnostic safety note.
11. Agentic write actions require confirmation.

Caregiver Mode

If the logged-in role is caregiver, the frontend adapts prompt wording.

Example elder prompt:

```
Did I miss any medicine this week?
```

Caregiver prompt:

```
Did your mother miss any medicine this week?
```

The backend still scopes data using the authenticated elder user id. The user_role is mainly used for wording and response style.

Failure Handling

The system handles failures safely:

```
Empty message -> rejected
ML-service unreachable -> friendly "try again" response
NL2SQL parse failure -> fallback response
Unsafe SQL -> "I cannot answer that safely"
SQL execution failure -> simpler-question fallback
Answer-generation failure -> raw retrieval fallback
Voice module unavailable -> user can still type
Microphone permission denied -> typed input remains available
```

This makes the assistant resilient even when AI or voice dependencies fail.

Why This Architecture Is Better Than a Direct LLM Chatbot

A direct LLM chatbot would be risky because it could hallucinate, expose data incorrectly, or generate unsupported medical guidance.

This architecture is safer because:

```
The frontend never calls the LLM directly
The backend controls identity and data access
The LLM sees only an approved schema digest
The SQL validator blocks unsafe queries
The database query is user-scoped
The final answer is based on retrieved rows
Sensitive fields are redacted
Actions require confirmation
```

So the LLM improves usability, while the backend preserves safety and control.

Final Technical Summary

The assistant chat screen is an AI-assisted healthcare interface built with React Native, Node/Express, PostgreSQL, FastAPI, and an LLM. The frontend manages the conversation UI, voice input, voice output, loading state, follow-up chips, and chat history. The backend authenticates the user, classifies the message, handles deterministic health workflows,

validates LLM-generated SQL, retrieves user-specific records, redacts sensitive data, saves conversation history, and returns the final response.

The RAG pipeline improves answer reliability by grounding the LLM in real application records. The LLM is used for flexible natural language understanding and readable response generation, but safety-critical responsibilities remain in deterministic backend code.